

# Relatório de Integração – Sprint 2

## Bah-Food: Plataforma de Delivery Gaúcho

|             |                                                             |
|-------------|-------------------------------------------------------------|
| Aluno       | Arthur Astolfi Cardoso                                      |
| Disciplina  | Lab. de Desenvolvimento de Aplicações Móveis e Distribuídas |
| Instituição | PUC Minas – Engenharia de Software                          |
| Data        | 22 de maio de 2026                                          |

### 1. Escolha da Ferramenta

O middleware orientado a mensagens escolhido para o Bah-Food foi o **RabbitMQ 3.13**, executado via Docker. A escolha se justifica por três razões principais:

- **Maturidade e suporte ao ecossistema Node.js:** a biblioteca `amqp-lib` oferece integração direta com o protocolo AMQP 0-9-1, protocolo nativo do RabbitMQ, sem camadas de abstração que ocultem o comportamento do broker.
- **Flexibilidade de roteamento:** o modelo de exchanges e filas do RabbitMQ permite evoluir o sistema para padrões mais complexos (pub/sub, roteamento por tópico) sem alterar os produtores já implementados.
- **Observabilidade:** o RabbitMQ disponibiliza uma Management UI integrada (<http://localhost:15673>) que permite visualizar em tempo real o estado das filas, exchanges, bindings e taxa de mensagens — recurso valioso tanto para desenvolvimento quanto para demonstração do funcionamento.

### 2. Padrão Utilizado

Foi adotado o padrão **Topic Exchange** do RabbitMQ, com o seguinte design:

| Componente     | Nome / Valor             |
|----------------|--------------------------|
| Exchange       | bahfood (tipo: topic)    |
| Routing key 1  | pedido.criado            |
| Routing key 2  | pedido.status.atualizado |
| Fila prestador | fila.prestador           |
| Fila cliente   | fila.cliente             |

O uso de um **topic exchange** com routing keys hierárquicas foi uma decisão deliberada de extensibilidade. Nas Sprints 3 e 4, quando os aplicativos Flutter precisarem receber eventos em tempo real, bastará adicionar novos consumidores (ou uma bridge WebSocket) que se inscrevam em padrões como `pedido.status.*` — sem qualquer alteração nos produtores existentes. Esse princípio está alinhado ao conceito de **Open/Closed** da arquitetura limpa: o sistema está aberto para extensão e fechado para modificação.

As mensagens são configuradas com `persistent: true`, garantindo que não sejam perdidas em caso de reinicialização do broker.

### 3. Integração com o Backend

---

A integração foi realizada em duas camadas:

#### Produtores (`src/messaging/producer.js`)

Chamados diretamente pela camada de serviço (`pedidoService.js`) após cada operação de escrita no banco de dados. Os dois momentos de publicação são:

1. **Criação de pedido** — após `pedidoRepository.createWithItems`, o evento `pedido.criado` é publicado com o payload completo do pedido (id, cliente, endereço, total, itens).
2. **Atualização de status** — após `pedidoRepository.updateStatus`, o evento `pedido.status.atualizado` é publicado com o novo estado e o prestador vinculado.

#### Consumidores (`src/consumers/`)

Inicializados no bootstrap da aplicação (`index.js`), antes de o servidor HTTP aceitar requisições. Cada consumidor declara sua fila, realiza o binding com o exchange e processa as mensagens de forma assíncrona, simulando o comportamento que os aplicativos móveis terão nas sprints seguintes.

### 4. Desafios Encontrados e Soluções

---

#### Conflito de porta com outro projeto

Ao tentar subir o container do RabbitMQ, a porta 5672 já estava ocupada por outro broker em execução na máquina de desenvolvimento. A solução adotada foi mapear a porta do host para 5673 (5673:5672 no docker-compose), isolando completamente o broker do Bah-Food sem interferir em outros projetos.

#### Credenciais do broker compartilhado

Antes do isolamento, o backend estava configurado com credenciais de outro projeto. Após a criação do container próprio, as credenciais foram redefinidas para o padrão `guest/guest` e a variável `RABBITMQ_URL` foi centralizada no `.env`, eliminando a dependência externa.

#### Ordem de inicialização

Para garantir que nenhuma publicação ocorra antes do canal AMQP estar pronto, a conexão com o RabbitMQ e a inicialização dos consumidores foram colocadas no fluxo assíncrono do `index.js`, antes da chamada `app.listen`. Isso assegura que o servidor HTTP só aceite requisições após toda a infraestrutura de mensageria estar operacional.

## 5. Conclusão

---

A integração do RabbitMQ ao Bah-Food estabelece a base da Arquitetura Orientada a Eventos do sistema. Os dois fluxos implementados — notificação de novo pedido ao prestador e atualização de status ao cliente — demonstram comunicação assíncrona real: o produtor publica o evento e retorna imediatamente ao chamador REST, enquanto o consumidor processa a mensagem de forma independente, sem acoplamento direto entre as partes. Essa separação é o alicerce sobre o qual os aplicativos Flutter serão construídos nas Sprints 3 e 4.